

Orchestration for efficient LLM Checkpointing within HPC

Advanced Computing Project 25/26

André Miranda PG60238

Gustavo Barros PG61527

José Soares PG60277

Miguel Carvalho PG61153

Department of Informatics – School of Engineering – University of Minho

April 24st 2026

Table of contents

- 1 Introduction
 - Context
 - Problem
 - Motivation
- 2 Approach and Architecture
 - Approach
 - Architecture
- 3 Results
- 4 Evaluation & Future Work
 - Next Steps

Context

LLM training spans several weeks, requiring periodic checkpointing to a Parallel File System (PFS) to prevent total state loss during node failures.

However, checkpointing directly to the PFS creates two problems:

- Simultaneous writes create heavy I/O contention, causing the "noisy neighbor" problem for other jobs.
- High write latency stalls training, leading to significant compute time being wasted.

Problem

Challenge	Impact
PFS Contention	I/O bottlenecks slow training as nodes compete for bandwidth.
Resilience	Hardware failures at scale risk losing weeks of training progress.
Checkpointing	Larger models → larger checkpoints → higher storage costs/time.
Storage Costs	Frequent PFS writes increase costs and operational overhead.

Table 1: Challenges of LLM Checkpointing to Parallel File Systems

Motivation

The goal is to design a checkpointing system that:

- Mitigates I/O contention on the PFS, ensuring fair resource sharing and Cluster Harmony.
- Provides resilience against hardware failures, minimizing training disruptions.
- Optimizes checkpointing frequency and storage strategies to balance performance and costs.

Approach: Policy-Driven Checkpointing

Controlled PFS Access

Mitigate I/O contention
via a centralized
Orchestrator.

Policy-Driven

Dynamic migration
based on cluster-wide
scheduling policies.

Traffic Shaping

Token Bucket to
smooth traffic and
eliminate noisy
neighbors.

Tiered Persistence

Local SSDs as a buffer,
decoupling training from
PFS latency.

Architecture

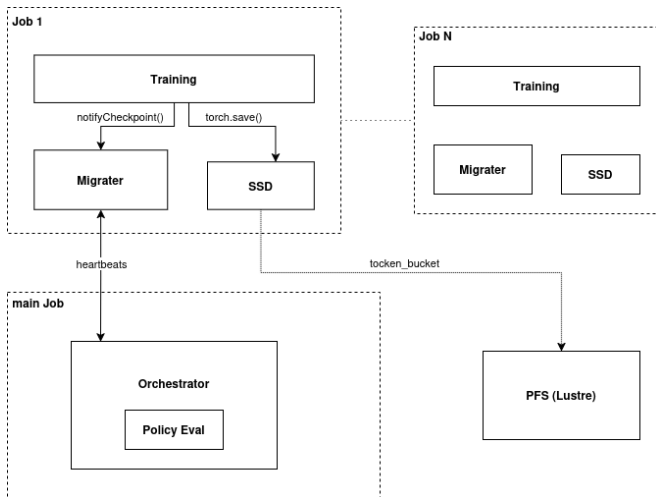


Figure 1. A sample image for the presentation.

Results

- TODO

Next Steps: Scaling and Benchmarking

Deucalion Deployment

Transition from local testing to the **Deucalion supercomputer** to validate scalability across multiple ARM nodes.

Policy Refinement

Evaluate different **scheduling policies** within the Orchestrator to find the optimal balance for PFS throughput.

Profiling

Extract performance data using the **PyTorch Profiler** during large-scale runs (already configured).

System Robustness

Ensure the Token Bucket performs reliably during massive concurrent transfers, preventing any bypass of the defined I/O policies.